Complexité algorithmique - Solution

Exercice 1 : Somme des n premiers nombres

1) Annoter le programme pour compter combien de fois chaque ligne est exécutée.

#	Programme	Opérations	Executions	Sous-total
1	`somme = 0`	1	1	1
2	`for i in range(n):`	1	1	1
3	`    somme = somme + (i + 1)`	3	n	$3n + 3$
4	`print(somme)`	1	1	1

2) Trouver la fonction qui lie le nombre d'opérations à n

$f(n) = 3n + 6$

3) Déterminer la classe de complexité du programme

$O(n)$

Exercice 2 : Somme des n premiers nombres – forme fermée

Il y a une manière plus rapide de calculer la somme des n premiers nombres, grâce à cette identité :

$$1 + 2 + 3 + \cdots + (n-1) + n = \frac{n \times (n+1)}{2}$$

1) Annoter le programme pour compter combien de fois chaque ligne est exécutée

#	Programme	Opérations	Executions	Sous-total
1	`somme = (n * (n + 1)) / 2`	4	1	4
2	`print(somme)`	1	1	1

2) Trouver la fonction qui lie le nombre d'opérations à n

$f(n) = 5$

3) Déterminer la classe de complexité du programme

$O(1)$

Exercice 3 : Table de multiplication

Le programme suivant affiche la table de multiplication de 1 à n.

1) Annoter le programme pour compter combien de fois chaque ligne est exécutée

#	Programme	Opérations	Executions	Sous-total
1	`for i in range(n):`	1	1	1
2	`    for j in range(n):`	1	n	n
3	`        print((i+1) * (j+1))`	4	n^2	$4n^2$

2) Trouver la fonction qui lie le nombre d'opérations à n

$f(n) = 4n^2 + n + 1$

3) Déterminer la classe de complexité du programme

$O(n^2)$

Exercice 4 : Déterminer si un nombre est premier

1) Annoter le programme pour compter combien de fois chaque ligne est exécutée

#	Programme	Opérations	Executions	Sous-total
1	`for i in range(2, n):`	1	1	1
2	`  if n % i == 0:`	2	$n-2$	$2n-4$
3	`    print(i, "divise", n)`	1	x	x
4	`  else:`	0	0	0
5	`    print(".")`	1	$n-2-x$	$n-2-x$

Rappel : `n % i` calcule le reste de la division entière de `n` par `i`.

2) Trouver la fonction qui lie le nombre d'opérations à n

$f(n) = 3n - 5$

3) Déterminer la classe de complexité du programme

$O(n)$